

Exercice Pratique AWS RDS

Objectif : Créer une base de données MySQL sur RDS et la connecter à votre instance EC2 via Node.js
Durée : 45-60 minutes

Configuration RDS

- Choisir MySQL comme moteur
- Version : MySQL 8.0.28
- Template : Free tier
- DB Instance Identifier : myapp-database
- Master username : admin
- Générer un mot de passe sécurisé
- Instance type : db.t3.micro

1. Sur l'EC2, installer le client MySQL et créer la base de données

Installation du client MySQL

```
sudo dnf install mariadb105 -y
```

Connexion à votre instance RDS

```
mysql -h [VOTRE-ENDPOINT-RDS] -P 3306 -u admin -p
```

Une fois connecté à MySQL, créer la base et la table

```
CREATE DATABASE webappdb;
```

```
USE webappdb;
```

Création de la table users

```
CREATE TABLE users (
```

```
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```
username VARCHAR(50) NOT NULL,  
email VARCHAR(100) NOT NULL,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Insertion de données de test

```
INSERT INTO users (username, email) VALUES
```

```
('john_doe', 'john@example.com'),  
('jane_doe', 'jane@example.com'),  
('bob_smith', 'bob@example.com');
```

Vérifier que tout est bien créé

```
SELECT * FROM users;
```

Quitter MySQL

```
exit
```

2. Sur votre instance EC2, installer Node.js

Installation de Node.js

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash
```

```
. ~/.nvm/nvm.sh
```

```
nvm install 16
```

```
nvm use 16
```

3. Créer un projet Node.js

```
mkdir myapp
```

```
cd myapp
```

```
npm init -y
```

Installation des dépendances

npm install express mysql2 dotenv

Créer le fichier server.js

```
require('dotenv').config();

const express = require('express');
const mysql = require('mysql2/promise');
const app = express();

const pool = mysql.createPool({
  host: votre-endpoint-rds,
  user: admin,
  password: votre-mot-de-passe,
  database: webappdb
});

app.get('/', async (req, res) => {
  try {
    const [rows] = await pool.query('SELECT * FROM users');
    res.send(`
      <h1>Liste des utilisateurs</h1>
      ${rows.map(user => `
        <div>
          <p>Utilisateur : ${user.username}</p>
          <p>Email : ${user.email}</p>
        </div>
      `).join("")}
    `);
  }
});
```

```
} catch (error) {  
  console.error(error);  
  res.status(500).send('Erreur serveur');  
}  
});  
  
app.listen(3000, () => {  
  console.log('Serveur en cours d\'exécution sur le port 3000');  
});
```

4. Lancer l'application

Lancer en arrière-plan

```
node server.js &
```

Noter le PID si vous devez arrêter le processus plus tard

Pour arrêter : kill <PID>

5. Vérifier que l'application fonctionne

Tester localement

```
curl http://localhost:3000
```

L'application sera accessible via

```
http://[IP-PUBLIQUE-EC2]:3000
```

⚠ N'oubliez pas :

D'ouvrir le port 3000 dans le Security Group de l'EC2

De sauvegarder le PID quelque part si vous devez arrêter le processus

Cette méthode est pour le développement/test uniquement, pas pour la production

🎓 Pour arrêter l'application :

Trouver le PID

```
ps aux | grep node
```

Arrêter l'application

kill <PID>